

Semana 12

SEMANA 12 · INTEGRACIÓN Y PRUEBAS DEL PROTOTIPO

En las semanas 12 y 13 avanzaremos la integración de VigiAire: planificación y seguimiento, estructura de referencia, circuito electrónico simulado en Wokwi, interfaz Android y pruebas de funcionamiento conjunto. Documentaremos con capturas, esquema de conexiones, código comentado y un video que muestre la lectura simultánea en el LCD y la app. Cada evidencia estará en un bloque independiente para incorporar imágenes y enlaces durante el desarrollo.

01 · PLANIFICACIÓN Y DIAGRAMA DE GANTT (SEMANAS 12 Y 13)

Inicio Desarrollo Prototipo final

Registrar las actividades, responsables, fechas y porcentaje de avance de ambas semanas. El cuadro de planificación y el Gantt deben reflejar los cambios reales del prototipo: maqueta de referencia, circuito simulado, app Android y pruebas de integración.

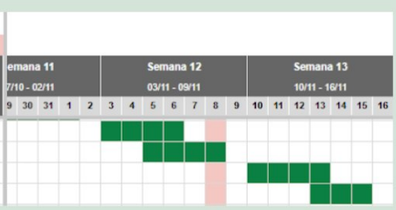
Curso: Investigación e Innovación Tecnológica
Docente: María Ángela Mejía León
Equipo:
Integrantes Vasquez Flores Luis Antonio, Franklin Gonzales Palomino, Jesus Castillo Sumire, Marko Osorio
Proyecto: VigiAire – prototipo virtual de monitoreo y alerta de aire (simulación)

SEMANA	FASE DE DESIGN THINKING	ACTIVIDAD PRINCIPAL	SUB TAREA	FECHA DE INICIO	FECHA DE FIN	DURACIÓN (DÍAS)	RESPONSABLE	SUPERVISOR	ESTADO
Semana 12	Prototipar	Actualizar el modelo 3D y el circuito simulado	Modificar el modelo 3D de VigiAire y revisar sus vistas frontal, lateral, superior e inferior; verificar también las conexiones en Wokwi.	03/11/2026	06/11/2026	4	Franklin	Jesus	Completo
Semana 12	Prototipar	Mejorar la lógica del circuito simulado	Ajustar la lectura del MQ-2, los umbrales, los mensajes LCD/LED y la salida serial; documentar que los valores son simulados.	05/11/2026	08/11/2026	4	Jesus	Marko	Completo
Semana 13	Prototipar	Implementar mejoras en la aplicación Android	Actualizar legibilidad, tendencia, navegación y alertas conforme a los hallazgos de las pruebas.	10/11/2026	13/11/2026	4	Luis	Jesus	Completo
Semana 13	Testear	Probar el flujo simulado completo	Repetir escenarios de lectura baja, media, alta y sin datos; verificar que la app y Wokwi presenten estados coherentes.	13/11/2026	15/11/2026	3	Marko	Luis	Completo

Diagrama de Gantt – VigiAire virtual y simulado

Leyenda: Completo | En proceso | Pendiente | Hoy

SEMANA	ACTIVIDAD PRINCIPAL	RESPONSABLE	ESTADO	INICIO	FIN
Semana 12	Actualizar el modelo 3D y el circuito simulado	Franklin	Completo	03/11/2026	06/11/2026
Semana 12	Mejorar la lógica del circuito simulado	Jesus	Completo	05/11/2026	08/11/2026
Semana 13	Implementar mejoras en la aplicación Android	Luis	Completo	10/11/2026	13/11/2026
Semana 13	Probar el flujo simulado completo	Marko	Completo	13/11/2026	15/11/2026



Inicio Desarrollo Prototipo final

02 · ACTUALIZACIÓN DEL MODELO 3D

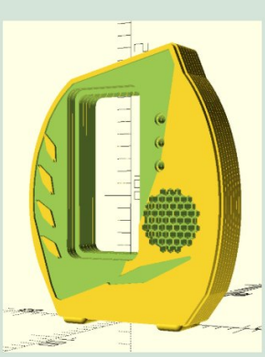
Actualización del modelo 3D de referencia

La propuesta de VigiAire evolucionó en tres etapas. **Antes:** una maqueta de caja representaba la forma inicial del dispositivo. **Durante:** se reunieron los materiales y se definieron los cambios en la distribución de sus elementos. **Después:** se elaboró un modelo 3D actualizado que muestra la forma propuesta desde las vistas frontal, lateral, superior e inferior.

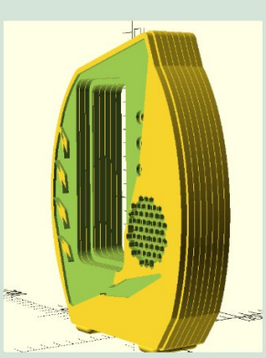
El modelo incorpora un espacio frontal para el teléfono, tres indicadores luminosos, una rejilla de ventilación y apoyos en la base. Estas vistas permiten evaluar la apariencia y la organización de los elementos antes de construir una nueva maqueta. El modelo representa una propuesta de diseño; todavía no demuestra el funcionamiento físico del dispositivo.

Link thinkercad: https://www.tinkercad.com/things/gqtccSMzAcV-swanky-turing/edit?returnTo=https%3A%2F%2Fwww.tinkercad.com%2Fdashboard%2Fdesigns%2Fall&sharecode=H-Yac2jbzWkFWMeGo34V6Y7Q0oNiQo0smPV59WgpNa8

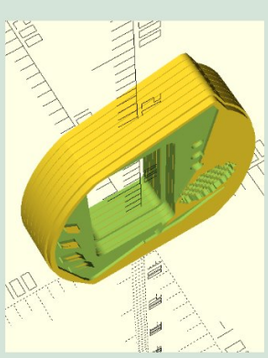
Modelado 3D realizado en OpenSCAD:



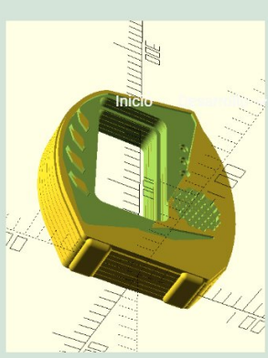
Vista frontal: Muestra el espacio previsto para el teléfono, los tres indicadores luminosos y la rejilla de ventilación. Su distribución permite identificar los elementos principales de VigiAire.



Vista lateral: Permite apreciar el grosor de la carcasa y sus capas, que aportan profundidad al diseño y delimitan el espacio interior.



Vista superior: Muestra la forma inclinada de la parte alta y la continuidad de las capas que conforman la carcasa.



Vista inferior: Permite observar la base y los apoyos previstos para mantener estable la maqueta sobre una superficie.

Inicio Prototipo final

03 · ESTRUCTURA DEL PROTOTIPO: AVANCE

Se elaboró una plantilla a escala para definir la forma y distribución frontal de la maqueta VigiAire. El diseño considera un espacio para colocar el celular, tres indicadores de estado y una abertura circular para representar la zona del sensor o ventilación. La plantilla se utilizó como guía para marcar y recortar el panel, y luego se ajustó la pieza obtenida. Este avance corresponde a la estructura visual del prototipo; las imágenes no muestran todavía la integración del circuito electrónico.

Materiales y herramientas utilizados

- **Plantilla impresa:** contiene el contorno y la distribución frontal del diseño.
- **Panel rígido:** base recortada siguiendo la plantilla. Por la apariencia de la veta podría ser una placa de madera o MDF/Trupan; confirma el material exacto antes de nombrarlo en el informe.
- **Cinta adhesiva:** sujeta la plantilla al panel durante el marcado y el corte.
- **Tijera o herramienta de corte:** se observa una herramienta manual para recortar el contorno.
- **Impresora:** utilizada para obtener la plantilla en papel.



Impresión de la plantilla

Se imprime el diseño frontal de VigiAire con sus medidas y las zonas previstas para el celular y los indicadores.



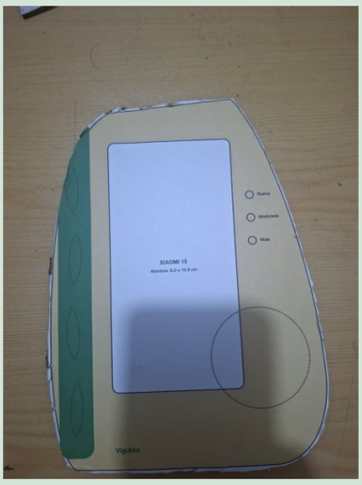
Fijación de la plantilla

La plantilla se coloca sobre el panel y se sujeta con cinta adhesiva para mantenerla alineada durante el marcado y el corte.



Corte del contorno

Se recorta el panel siguiendo el perímetro de la plantilla para obtener la forma exterior de la estructura.



Ajuste de la pieza frontal

Se observa la pieza recortada con el diseño frontal y sus aberturas de referencia para el celular, los indicadores y la zona circular.



Panel posterior o base

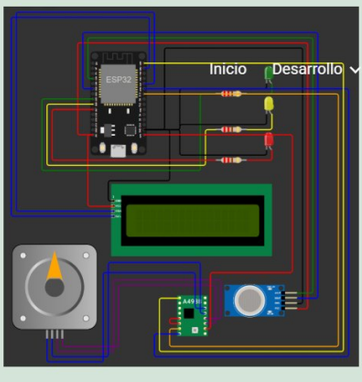
Se muestra el reverso de la pieza estructural. Esta cara sirve como soporte de la maqueta y todavía no presenta acabados ni componentes electrónicos integrados.

04 · COMPONENTES ELECTRÓNICOS Y CIRCUITO

El circuito de VigiAire se simula en Wokwi con un ESP32, un sensor MQ-2, una pantalla LCD, tres LEDs con resistencias, un controlador A4988 y un motor paso a paso bipolar. El ESP32 procesa la señal ADC simulada, muestra el estado en la pantalla y los indicadores, y envía la lectura a la aplicación mediante Wi-Fi y MQTT. Desde la app también se puede activar o detener el giro simulado del motor. Este movimiento es demostrativo: no produce succión ni mide la calidad real del aire.

Enlace del proyecto Wokwi:
https://wokwi.com/projects/478264195609475073

VISTA GENERAL DEL CIRCUITO EN WOKWI:



LISTA DETALLADA DE COMPONENTES:

COMPONENTE	CANTIDAD	FUNCIÓN Y CONEXIONES
ESP32 DevKit V1	1	Unidad de control. Lee la señal ADC del MQ-2, determina VENTILE, ESPERE o ALERTA, actualiza LCD y LEDs, publica datos por MQTT y recibe la orden de giro del motor.
Sensor de gas MQ-2	1	Entrada simulada de gases/humo. VCC a VIN, GND a GND, AOUT a GPIO 34 y DOUT a GPIO 32. Los valores son señales simuladas, no mediciones ambientales reales.
Pantalla LCD 16 × 2 I²C	1	Muestra la lectura ADC y el estado. SDA a GPIO 21, SCL a GPIO 22, VCC a VIN y GND a GND.

LED verde	1	Indica VENTILE. Ánodo desde GPIO 25 a través de una resistencia de 220 Ω; cátodo a GND.
LED amarillo	1	Indica ESPERE. Ánodo desde GPIO 26 a través de una resistencia de 220 Ω; cátodo a GND.
LED rojo	1	Indica ALERTA. Ánodo desde GPIO 27 a través de una resistencia de 220 Ω; cátodo a GND.
Resistencias de 220 Ω	3	Una en serie con cada LED para limitar la corriente y proteger sus componentes.
Controlador A4988	1	Controla el motor paso a paso: STEP a GPIO 18, DIR a GPIO 19 y ENABLE a GPIO 23. VDD a 3V3, GND común; RESET y SLEEP unidos.
Motor paso a paso bipolar	1	Gira en la simulación al activar el interruptor de la app. Sus bobinas se conectan a 1A, 1B, 2A y 2B del A4988. No produce succión ni mueve aire.
Cables de conexión	**27**	**27 conexiones individuales según el diagrama actual de Wokwi: alimentación, tierra, señales, LEDs, controlador y bobinas del motor.**

05 · CONEXIONES, PINES Y ALIMENTACIÓN

El circuito simulado conecta el ESP32 con el sensor MQ-2, la pantalla LCD, los indicadores LED y el controlador A4988. En la siguiente tabla se detallan los pines de señal y las conexiones de alimentación de cada módulo, incluyendo la referencia común a GND.

ELEMENTO	PINES Y SEÑALES	ALIMENTACIÓN / CONEXIÓN
ESP32 DevKit V1	GPIO 34 recibe AOUT del MQ-2; GPIO 32 recibe DOUT; GPIO 21 y 22 controlan SDA y SCL del LCD; GPIO 25, 26 y 27 controlan los LEDs; GPIO 18, 19 y 23 controlan STEP, DIR y ENABLE del A4988.	VIN alimenta el MQ-2 y el LCD; 3V3 alimenta la lógica del A4988; GND es la referencia común del circuito.
Sensor MQ-2	AOUT → GPIO 34; DOUT → GPIO 32.	VCC → VIN del ESP32; GND → GND común.
LCD 16 × 2 I²C	SDA → GPIO 21; SCL → GPIO 22.	VCC → VIN del ESP32; GND → GND común.
LEDs y resistencias	GPIO 25 → resistencia 220 Ω → ánodo LED verde; GPIO 26 → resistencia 220 Ω → ánodo LED amarillo; GPIO 27 → resistencia 220 Ω → ánodo LED rojo. Cada cátodo va a GND.	Una resistencia de 220 Ω en serie con cada LED. Los tres LEDs comparten GND con el ESP32.
Controlador A4988	STEP → GPIO 18; DIR → GPIO 19; ENABLE → GPIO 23; RESET y SLEEP están unidos.	VDD → 3V3 del ESP32; GND → GND común.
Motor paso a paso bipolar	1B → B−; 1A → B+; 2A → A+; 2B → A− del A4988.	Las bobinas se conectan a las salidas del A4988; el motor no se conecta directamente a los GPIO del ESP32.

El ESP32 lee la señal analógica del MQ-2 por el GPIO 34 y determina el estado mostrado en el LCD y los LEDs. La app envía por Wi-Fi y MQTT la orden para activar o detener el giro simulado: el ESP32 controla el A4988 mediante STEP, DIR y ENABLE, y este acciona el motor paso a paso. La simulación sirve para visualizar las conexiones y la lógica de control; no mide la calidad real del aire ni genera succión.

06 · PROGRAMACIÓN DE LA INTERFAZ DIGITAL

Se presenta la interfaz digital de VigiAire desarrollada en Android Studio y sus principales pantallas. Las capturas muestran las funciones implementadas y los cambios realizados, acompañados de una breve descripción de su propósito. También se incluye el flujo de navegación para explicar cómo se relacionan las secciones de la aplicación y cómo se comunica con el circuito simulado en Wokwi.

Repositorio GitHub: https://github.com/JesusCastillo2026/Investigacion-Innovacion-C24-Grupo.git



Pantalla de bienvenida

Presenta la identidad de VigiAire y explica brevemente su propósito: mostrar lecturas simuladas y ayudar a comprender los estados del sistema. El botón «Comenzar» lleva al usuario al inicio de sesión.



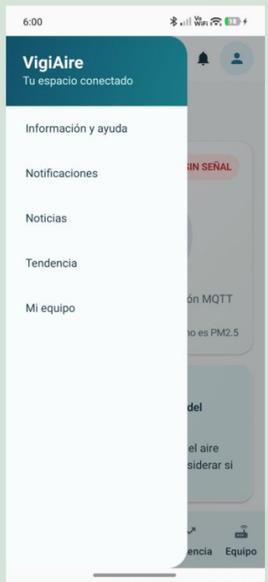
Inicio de sesión

Muestra el acceso a la aplicación mediante correo electrónico y contraseña. También incluye una cuenta de demostración para que el usuario pueda explorar las funciones de VigiAire.



Tutorial inicial

Orienta al usuario sobre cómo iniciar la simulación en Wokwi, interpretar las lecturas y recorrer las secciones principales. El tutorial aparece al comenzar una nueva ejecución de la aplicación.

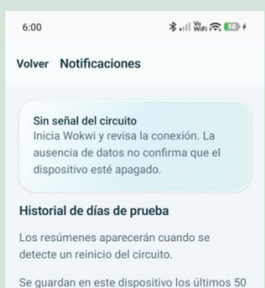


Menú de navegación

Presenta un panel lateral para acceder a las secciones de ayuda, notificaciones, noticias, tendencia y equipo. Así, el usuario puede encontrar cada función desde un solo lugar.











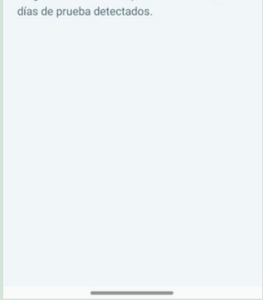
Guía de VigiAire

Presenta los rangos de la señal simulada del sensor MQ-2 y explica los estados VENTILE, ESPERE y ALERTA. Las preguntas desplegables ayudan a entender qué significa el valor ADC y cómo interpretar cada lectura.



Mi dispositivo

Muestra el estado de conexión con el circuito Wokwi y permite conectar o desconectar la app. También incluye un interruptor para iniciar o detener el giro simulado del motor paso a paso y consultar la última lectura recibida.



Notificaciones

Informa cuando no se reciben señales del circuito y orienta al usuario para revisar la conexión y la simulación en Wokwi. También presenta el historial de días de prueba detectados y los resúmenes disponibles.



Mi perfil

Muestra la información de demostración del usuario y ofrece opciones de personalización, como activar el modo noche, ampliar el texto y reforzar el contraste para facilitar la lectura.

Flujo de navegación de la app

La tabla muestra el recorrido principal de VigiAire, desde la bienvenida y el inicio de sesión hasta las secciones de consulta y configuración. También resume las condiciones para conectar la app con Wokwi y activar o detener el giro simulado del motor.

PANTALLA O ACCIÓN	INTERACCIÓN	RESULTADO / CONDICIÓN
BIENVENIDA	Pulsa «Comenzar».	Abre la pantalla de inicio de sesión.
INICIO DE SESIÓN	Ingresa con la cuenta de demostración.	Accede a Inicio. El tutorial aparece una vez por cada ejecución de la app.
INICIO	Usa el menú lateral o la barra inferior.	Navega a Ayuda, Notificaciones, Noticias, Detalle, Consejo, Tendencia o Equipo.
MI DISPOSITIVO	Pulsa «Conectar dispositivo».	La app intenta conectarse al circuito Wokwi mediante MQTT.
CONTROL DEL MOTOR	Activa o desactiva el interruptor.	Envía la orden de inicio o detención del giro simulado. Requiere conexión y una lectura reciente.
MI PERFIL	Cambia modo noche, tamaño de texto o contraste.	Personaliza la presentación de la interfaz.
CERRAR SESIÓN	Pulsa «Cerrar sesión» desde el perfil.	Regresa al inicio de sesión. El tutorial no reaparece hasta reiniciar la app.

07 · CÓDIGO Y VINCULACIÓN INALÁMBRICA

El ESP32 procesa la señal simulada del sensor MQ-2 y publica por Wi-Fi, mediante MQTT, la lectura ADC, el estado del sistema y los datos de sesión. La aplicación Android se suscribe al canal de telemetría para recibir y presentar esos datos. Cuando el usuario activa o detiene el giro simulado del motor desde la app, esta envía una orden MQTT al ESP32. El LCD y la aplicación muestran la información publicada por el circuito; puede existir un breve retraso de comunicación. Las lecturas y el movimiento se realizan en una simulación.


```
void publishReading(int value, const char* state) {
  char payload[220];
  snprintf(payload, sizeof(payload),
           R"json({"device":"VigiAire-ESP32","sensor":"MQ-2","simulated":true,"gasAdc":%d,
           "status":"%s","extractorEnabled":%s,"seq":%lu,"uptimeMs":%lu})json",
           value, state, extractorRequested ? "true" : "false", ++sequenceNumber, millis());
  Serial.println(mqtt.publish(MQTT_TOPIC, payload, false)
      ? "MQTT publicado correctamente."
      : "No se pudo publicar en MQTT.");
}
```

1. Publicación de lecturas por MQTT
La función `publishReading` prepara un mensaje con el valor ADC simulado del sensor MQ-2, el estado calculado —VENTILE, ESPERE o ALERTA—, el estado del control del motor y datos de la sesión. Luego publica esa información mediante MQTT para que la aplicación Android pueda recibirla y mostrarla. Así, la lectura que presenta la app corresponde a los datos enviados por el circuito simulado.

```
void onMqttMessage(char* topic, byte* payload, unsigned int length) {
  if (strcmp(topic, MQTT_COMMAND_TOPIC) != 0) return;
  String body;
  body.reserve(length);
  for (unsigned int i = 0; i < length; i++) body += static_cast<char>(payload[i]);
  if (body.indexOf("\"command\":\"extractor\"") < 0) return;
  extractorRequested = body.indexOf("\"enabled\":true") >= 0;
  Serial.println(extractorRequested ? "Motor paso a paso activado desde la app."
                                    : "Motor paso a paso detenido desde la app.");
}
```

2. Recepción de órdenes para el motor
La función `onMqttMessage` escucha los mensajes recibidos por el ESP32 y verifica que la orden provenga del tema MQTT configurado para los comandos. Si el mensaje solicita activar o detener el motor, actualiza el estado correspondiente. De esta manera, la orden enviada desde la app puede llegar al ESP32 para controlar el motor paso a paso en Wokwi.

```
mqttClient.connect(options, userContext = null, callback = object : IMqttActionListener {
    override fun onSuccess(asyncActionToken: org.eclipse.paho.client.mqttv3.IMqttToken?) {
        _state.value = MqttConnectionState.CONNECTED
    }
```

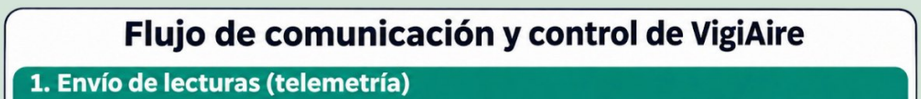
3. Conexión de la aplicación al broker MQTT
Este fragmento inicia la conexión de la app Android con el broker MQTT. Cuando la conexión se establece correctamente, la aplicación actualiza su estado y queda lista para intercambiar información con el circuito. Esta conexión es necesaria tanto para recibir las lecturas como para enviar órdenes de control.

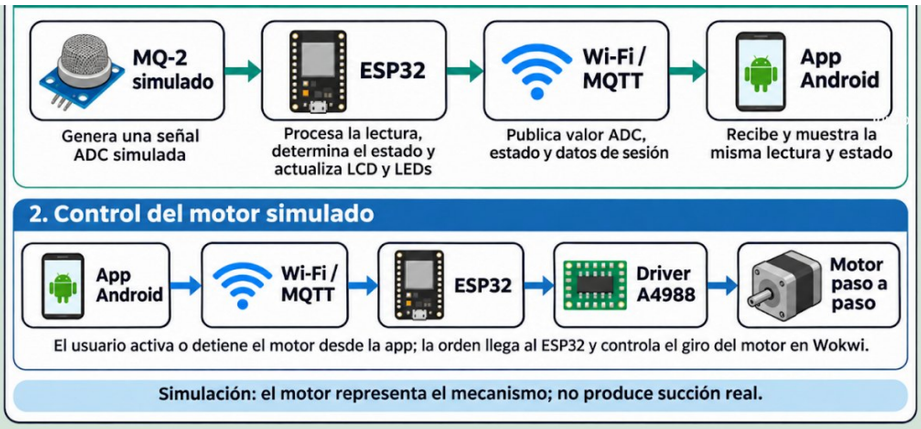

```
private fun subscribe(mqttClient: MqttAsyncClient) {
    if (!mqttClient.isConnected) return
    mqttClient.subscribe( topicFilter = TOPIC, qos = 0, userContext = null, callback = object : IMqttActionListener {
        override fun onSuccess(asyncActionToken: org.eclipse.paho.client.mqttv3.IMqttToken?) {
            _error.value = null
        }
```

4. Suscripción al tema de telemetría
Una vez conectada, la app se suscribe al tema MQTT donde el ESP32 publica las lecturas. La suscripción permite recibir automáticamente los mensajes nuevos y actualizar la información visible en la interfaz, como el valor ADC, el estado del sensor y los datos de la sesión.

```
fun setExtractorEnabled(enabled: Boolean) {
    val mqttClient = client
    if (_state.value != MqttConnectionState.CONNECTED || mqttClient?.isConnected != true) {
        _error.value = "Conecta la app y mantén Wokwi en ejecución para controlar la extracción."
        return
    }
```

5. Envío del comando desde la app
La función `setExtractorEnabled` gestiona la orden de activar o detener el motor desde la aplicación. Antes de enviar el comando, comprueba que exista conexión MQTT; si no la hay, muestra un aviso para conectar la app y mantener Wokwi en ejecución. Cuando la conexión está disponible, la app puede publicar la orden para que el ESP32 controle el motor simulado.





08 · PRUEBA ELECTRÓNICA Y FUNCIONAMIENTO CONJUNTO

Se verificará el recorrido completo de los datos: el MQ-2 genera una señal simulada, el ESP32 la procesa y muestra el valor y el estado en el LCD y en la app. También se comprobará la orden de activar o detener el motor paso a paso desde la app. Las pruebas se realizarán en Wokwi; el motor representa el mecanismo y no produce succión real.



App y circuito en Wokwi

La pantalla de VigiAire recibe mediante MQTT la lectura ADC y el estado procesados por el ESP32. En paralelo, el LCD del circuito muestra esos mismos datos. El monitor serial permite comprobar que los mensajes se publican correctamente y que la información llega a la aplicación. La lectura es simulada y sirve para demostrar la comunicación entre los componentes.



Resumen de la sesión

La aplicación calcula y presenta el valor mínimo, el promedio y el máximo de las lecturas recibidas durante la sesión. El indicador de pico resalta el valor más alto para facilitar su identificación. El gráfico se actualiza con los nuevos mensajes y reinicia la serie cuando detecta que el ESP32 volvió a iniciar. Estos datos corresponden a la señal simulada del MQ-2; no son mediciones reales de PM2.5 ni de calidad del aire.

LINK DEL VIDEO SOBRE INTERACCION FISICA:

- https://drive.google.com/drive/u/1/folders/1oFMlvn6epzqx0qYz6jHW9s5I0cnAOuVx

09 · RESULTADOS, ACIERTOS Y MEJORAS

Resultados de la prueba: Se integró la señal simulada del MQ-2 con el ESP32. El valor ADC y el estado se muestran en el LCD y se envían por MQTT a la app, donde también se visualizan y se resumen mediante valores mínimo, promedio y máximo. Además, desde la app se puede enviar la orden de activar o detener el motor paso a paso simulado en Wokwi.

Aciertos: Se logró comunicar el circuito y la aplicación, mostrar lecturas coherentes en ambas interfaces y organizar la información para que sea fácil de interpretar. También se incorporó el control remoto del motor como representación visual del mecanismo de extracción.

Oportunidades de mejora y siguiente paso: Conviene continuar verificando la sincronización entre la app, el LCD y el circuito, y mejorar la presentación de las lecturas y los estados. El MQ-2 y el motor funcionan como elementos simulados en Wokwi: no miden la calidad real del aire ni generan succión física.

10 · FUENTES, USO DE IA Y ENTREGA

Se documentan las fuentes consultadas y las herramientas utilizadas durante el desarrollo. La IA se empleó como apoyo de asesoría para organizar el Google Site y revisar que el contenido respondiera a los requisitos indicados en la guía del laboratorio. La entrega incluye el enlace al Site publicado y el PDF correspondiente.

Uso de inteligencia artificial:

Se utilizó ChatGPT como herramienta de asesoría y guía para organizar el contenido del Google Site y contrastar el avance con los requisitos del laboratorio. Se consultaron los modelos GPT-6 Astra y GPT-6 Luna para apoyar la revisión y planificación del trabajo.

Fuentes y enlaces:

- **Excel:** https://drive.google.com/file/d/11hRr0Goawp43Ymr9Ix_Dju2aWj0eOJSr/view?usp=sharing
- **Repositorio GitHub:** https://github.com/JesusCastillo2026/Investigacion-Innovacion-C24-Grupo
- **Video:** https://drive.google.com/drive/u/1/folders/1oFMlvn6epzqx0qYz6jHW9s5I0cnAOuVx
- **Wokwi:** https://wokwi.com/projects/478264195609475073
- **Modelalo 3D:** https://www.tinkercad.com/things/gqtccSMzAcV-swanky-turing/edit?returnTo=https%3A%2F%2Fwww.tinkercad.com%2Fdashboard%2Fdesigns%2Fall&sharecode=H-Yac2jbzWkFWMeGo34V6Y7Q0oNiQo0smPV59WgpNa8